Machine Learning Practical Exam

Viva Questions and Answers for all 11 practicals | 394 questions in total

1	Feature Engineering (Missing Values, Encoding, Scaling)	30 Q&A
2	Dimensionality Reduction with PCA and LDA	34 Q&A
3	Multiple Linear Regression (MSE, RMSE, R^2)	36 Q&A
4	Polynomial Regression and the Bias-Variance Trade-off	36 Q&A
5	Regularisation with Ridge and Lasso Regression	35 Q&A
6	Logistic Regression, Confusion Matrix, Precision, Recall and F1-score	41 Q&A
7	Support Vector Machines with Linear and Non-linear Kernels	36 Q&A
8	K-Means Clustering (Elbow Method and Silhouette Score)	34 Q&A
9	Density-Based Clustering with DBSCAN	32 Q&A
10	Ensemble Learning (Random Forest and Boosting) vs a Baseline Model	37 Q&A
11	Reinforcement Learning with Q-Learning (Grid World)	43 Q&A

How to use: in each practical, **A** = concept questions, **B** = questions about the graphs, **C** = code questions (labelled Q, G and C). Where an answer mentions numbers (accuracy, R^2, etc.), they come from the author's own run of the notebook: your numbers will differ slightly, so explain what *your* output shows.

Practical 1: Feature Engineering (Missing Values, Encoding, Scaling)

A. Concept questions

Q1. What is feature engineering?
Creating, transforming and selecting input variables so a model can learn better. It includes cleaning, handling missing values, encoding, scaling, creating new features and selecting useful ones.

Q2. Why do we handle missing values?
Most algorithms cannot work with NaN and raise errors. Ignoring them can also bias results.

Q3. What are the ways to handle missing values?
Delete rows or columns, fill with mean/median/mode, fill with a constant, forward/backward fill, or model-based imputation (KNN imputer, iterative imputer).

Q4. When do you use mean, median and mode?
Mean for symmetric numeric data. Median for skewed data or data with outliers (age, income). Mode for categorical data.

Q5. Why did you drop the `deck` column?
About 77% of its values are missing. Filling that many values would invent information, so dropping it is safer.

Q6. Why did you use the median for `age`?
Age has outliers and is slightly skewed, and the median is not affected by extreme values, unlike the mean.

Q7. What are the types of missing data?
MCAR (missing completely at random), MAR (missing at random, depends on other observed variables) and MNAR (missing not at random, depends on the missing value itself).

Q8. What is the difference between label encoding and one-hot encoding?
Label encoding assigns an integer to each category (0, 1, 2) and suits ordinal or binary data. One-hot creates a separate 0/1 column per category and suits nominal data because it does not imply any order.

Q9. What is the dummy variable trap?
If you keep all one-hot columns, one column is a perfect combination of the others (perfect multicollinearity). `drop_first=True` removes one column to avoid this.

Q10. What is ordinal encoding?
Mapping ordered categories to integers that keep the order, for example Low=0, Medium=1, High=2.

Q11. Why is scaling needed?
Features with large ranges (fare in hundreds) can dominate features with small ranges (age). Distance-based and gradient-based algorithms (KNN, SVM, K-Means, PCA, logistic regression, neural networks) need scaled data.

Q12. Difference between standardisation and normalisation?
Standardisation: $z = (x - \mu)/\sigma$, giving mean 0 and std 1. Normalisation (min-max): $x' = (x - x_{min})/(x_{max} - x_{min})$, giving values in [0, 1].

Q13. Which algorithms do not need scaling?
Tree-based models: Decision Tree, Random Forest, Gradient Boosting.

Q14. How do outliers affect scaling?
Min-max scaling is very sensitive to outliers (one huge value squeezes the others). StandardScaler is less affected. RobustScaler uses median and IQR and is the most robust.

Q15. What is data leakage?
When information from the test data influences training, for example fitting the scaler on the whole dataset before splitting. It gives falsely good results.

Q16. What is the difference between `fit`, `transform` and `fit_transform`?
`fit` learns parameters (mean, std), `transform` applies them, `fit_transform` does both in one step. Use `fit_transform` on train and only `transform` on test.

Q17. What is a categorical vs numerical variable?
Categorical holds labels or groups (sex, city). Numerical holds measurable numbers (age, fare); it can be discrete (count) or continuous (height).

Q18. How do you detect missing values in pandas?
`df.isnull().sum()` gives counts per column, `df.info()` shows non-null counts, and `sns.heatmap(df.isnull())` shows them visually.

Q19. What is the difference between feature selection and feature extraction?
Selection keeps a subset of original features. Extraction creates new features from the old ones (for example PCA).

Q20. What is a pipeline and why is it useful?
`sklearn.pipeline.Pipeline` chains steps (imputer, encoder, scaler, model) so they run in order and automatically avoid leakage.

B. Questions about the graphs

G1. What does the missing-values heatmap show?
Rows are samples, columns are features, and yellow marks missing cells. It shows which columns have many gaps (`deck` almost fully missing, `age` scattered) and whether missingness is concentrated.

G2. What does the boxplot of `age` and `fare` before scaling tell you?
The two features have very different ranges (fare up to about 500, age up to 80), and `fare` has many outliers (dots above the whisker). That is why scaling is needed.

G3. What does the histogram of `fare` show?
It is right-skewed: most passengers paid a low fare, and a few paid very high amounts. For such data the median is a better centre than the mean, and a log transform can reduce skew.

G4. What changed in the boxplots after scaling?
Both features now sit on a comparable scale (StandardScaler is centred at 0, MinMax is in [0, 1]), but the shape of the distribution and the outliers are unchanged. Scaling changes range, not shape.

G5. Does scaling remove outliers?
No. It only rescales the values; outliers remain.

C. Code questions

C1. What does `df['age'].fillna(df['age'].median())` do?
It replaces every NaN in `age` with the median of that column.

C2. Why `mode()[0]`?
`mode()` returns a Series (there can be ties), so `[0]` takes the first most frequent value.

C3. What does `pd.get_dummies(..., drop_first=True)` do?
It one-hot encodes the chosen columns and drops the first category of each, avoiding the dummy variable trap.

C4. Why `stratify=y` in `train_test_split`?
It keeps the same proportion of survived and not-survived in both train and test sets.

C5. Why scale only `age` and `fare` and not the 0/1 columns?
Binary columns are already in [0, 1], so scaling gives no benefit and makes them harder to interpret.

Practical 2: Dimensionality Reduction with PCA and LDA

A. Concept questions

Q1. What is dimensionality reduction? Why use it?
Reducing the number of input features while keeping most of the information. It reduces computation, storage, noise and overfitting, helps visualisation and fights the curse of dimensionality.

Q2. What is the curse of dimensionality?
As the number of features grows, data becomes sparse, distances lose meaning and models need exponentially more data, so they overfit easily.

Q3. What is PCA?
Principal Component Analysis is an unsupervised technique that transforms correlated features into a smaller set of uncorrelated variables (principal components), ordered by how much variance they capture.

Q4. Explain the steps of PCA.
(1) Standardise the data. (2) Compute the covariance matrix. (3) Find eigenvalues and eigenvectors. (4) Sort eigenvectors by eigenvalue, largest first. (5) Select the top k. (6) Project the data onto them.

Q5. What are eigenvalues and eigenvectors in PCA?
An eigenvector is a direction (a principal component). Its eigenvalue says how much variance lies along that direction.

Q6. Why must data be scaled before PCA?
PCA maximises variance, so features with large units dominate. Scaling gives all features equal weight.

Q7. What is LDA?
Linear Discriminant Analysis is a supervised technique that finds axes that maximise separation between classes (between-class variance) while minimising spread inside each class (within-class variance).

Q8. Difference between PCA and LDA?
PCA is unsupervised and maximises total variance; it can give up to n_features components. LDA is supervised and maximises class separability; it gives at most (number of classes - 1) components.

Q9. Why is the maximum number of LDA components c - 1?
With c classes the c class means lie in a space of dimension at most c - 1, so there are only c - 1 useful discriminant directions. A binary problem gives 1 component.

Q10. Is PCA feature selection?
No. It is feature extraction: new features are created as combinations of the old ones. Feature selection keeps a subset of original features.

Q11. How do you choose the number of components?
Keep enough to explain about 95% of the variance (cumulative plot), use the scree-plot elbow, or choose by validation accuracy.

Q12. Did performance fall after reduction? Why?
Slightly or not at all, because some information is discarded but most of it was redundant. The trade-off is lower computation and a simpler model for a small accuracy change.

Q13. Are principal components interpretable?
Not easily. Each is a mix of all original features. The weights (loadings) are in `pca.components_`.

Q14. What assumptions does LDA make?
Features are normally distributed within each class, all classes share the same covariance matrix, and classes are linearly separable.

Q15. Is PCA affected by outliers?
Yes, because it relies on variance and the covariance matrix, so extreme points can pull the components.

Q16. Are principal components correlated with each other?
No. They are orthogonal (uncorrelated) by construction.

Q17. What is explained variance ratio?
The fraction of total variance captured by each component. They sum to 1 over all components.

Q18. Can PCA be used for supervised problems?
Yes, as a preprocessing step before a classifier, but PCA itself ignores labels, so the top components are not guaranteed to be the most predictive.

Q19. What is the difference between LDA and logistic regression?
Both give linear boundaries. LDA is generative (models each class as a Gaussian with shared covariance) and is also used for dimensionality reduction. Logistic regression is discriminative and models the probability directly.

Q20. Give other dimensionality reduction methods.
t-SNE, UMAP, Kernel PCA, Autoencoders, SVD, Factor Analysis, and feature selection methods (filter, wrapper, embedded).

B. Questions about the graphs

G1. What does the cumulative explained variance plot show?
X-axis: number of components. Y-axis: share of total variance captured together. It rises steeply then flattens. About 10 components keep roughly 95% of the variance, so 30 features can be cut to about 10 with little loss.

G2. What is the dashed line at 0.95?
A threshold. The first x-value where the curve crosses it is the number of components to keep.

G3. What is a scree plot and how do you use it?
A plot of variance explained by each individual component. Look for the elbow where bars drop off and keep the components before it.

G4. What do PC1 and PC2 on the scatter plot mean?
PC1 is the direction of maximum variance in the data; PC2 is the next direction, perpendicular to PC1. Both are linear combinations of the original features.

G5. The two classes are separated in the PCA scatter. Why, if PCA ignores labels?
The classes differ strongly along the directions of highest variance, so even 2 components carry most of the class information. If the points overlapped heavily, 2 components would be insufficient.

G6. Why does the LDA plot separate classes better than PCA?
LDA uses labels and picks the axis maximising between-class distance relative to within-class spread. PCA only maximises total variance.

G7. What does the LDA histogram overlap represent?
The region where the two classes cannot be distinguished along LD1. Samples in that region are the likely misclassifications.

G8. What does the accuracy-vs-components graph show?
Accuracy rises quickly with the first few components and then plateaus near the baseline. More components after the plateau add cost but no benefit.

G9. Why is the accuracy bar chart x-axis starting at 0.85?
To zoom into small differences. Mention that this exaggerates differences, so read the numbers.

G10. What does the loadings table tell you?
Which original features contribute most to each principal component (large absolute value = strong influence).

C. Code questions

C1. Why `fit_transform` on train but only `transform` on test?
To avoid data leakage. The mean and std must come from training data only.

C2. What does `PCA(n_components=0.95)` do?
It automatically keeps the smallest number of components whose cumulative explained variance is at least 95%.

C3. Why does LDA need `y` in `fit` while PCA does not?
LDA is supervised and uses class labels; PCA is unsupervised.

C4. Why `stratify=y`?
To keep the class proportions the same in train and test sets.

Practical 3: Multiple Linear Regression (MSE, RMSE, R²)

A. Concept questions

Q1. What is linear regression?
A supervised algorithm that models a continuous target as a linear combination of the input features by fitting a straight line (or hyperplane).

Q2. What is multiple linear regression? Write the equation.
Regression with two or more features: $y = b_0 + b_1x_1 + b_2x_2 + \cdots + b_nx_n + \varepsilon$, where b_0 is the intercept and $b_1 \dots b_n$ are coefficients.

Q3. Difference between simple and multiple linear regression?
Simple has one input feature; multiple has two or more.

Q4. How are the coefficients found?
By Ordinary Least Squares, which minimises the sum of squared residuals. Closed form: $\beta = (X^TX)^{-1}X^Ty$. Gradient descent can also be used.

Q5. What does a coefficient mean?
The expected change in the target for a one-unit increase in that feature, with all other features held constant.

Q6. What is the intercept?
The predicted value when all features are zero (here, since the features are centred, it is the average target).

Q7. Explain MSE, RMSE and MAE.
MSE is the average of squared errors (penalises big errors, units squared). RMSE is its square root (same units as target, easy to interpret). MAE is the average absolute error (less sensitive to outliers).

Q8. Explain R².
The proportion of variance in the target explained by the model. 1 is perfect, 0 means no better than predicting the mean, and it can be negative for a very bad model.

Q9. What is adjusted R² and why use it?
R² never decreases when you add features, even useless ones. Adjusted R² penalises extra features, so it increases only if a new feature really helps.

Q10. What are the assumptions of linear regression?
Linearity, independence of errors, homoscedasticity (constant error variance), normally distributed errors, and no multicollinearity. Memory aid: **LINE** (Linearity, Independence, Normality, Equal variance).

Q11. What is multicollinearity and how do you detect it?
When features are highly correlated with each other, making coefficients unstable. Detect with the correlation matrix or VIF (above 5 or 10 is a concern). Fix by dropping features, PCA, or Ridge regression.

Q12. What is the difference between RMSE and R²?
RMSE is an absolute error in the target's units (lower is better). R² is a relative, unitless measure of fit (higher is better).

Q13. Why do we use squared error and not absolute error?
Squared error is smooth and differentiable, gives a unique closed-form solution, and penalises large errors more.

Q14. Is linear regression sensitive to outliers?
Yes, because squared errors give outliers a large pull on the line.

Q15. Is scaling needed for linear regression?
Not for the OLS solution itself, but it helps for gradient descent, regularised models and for comparing coefficient sizes.

Q16. How do you improve a weak R^2?
Add or engineer features, handle outliers, use polynomial features, use regularisation, or try non-linear models such as Random Forest.

Q17. What is heteroscedasticity?
When the variance of the errors changes across the predicted values (funnel shape in the residual plot). It violates the equal-variance assumption.

Q18. What is the difference between regression and classification?
Regression predicts a continuous number; classification predicts a category.

Q19. What are the problems if train R^2 is high but test R^2 is low?
Overfitting: the model memorised the training data. Remedies: regularisation, fewer features, more data, cross-validation.

Q20. What is the "best fit line"?
The line that minimises the sum of squared vertical distances between the points and the line.

Q21. Can R^2 be negative?
Yes, on test data, if the model predicts worse than simply using the mean of the target.

Q22. What is the normal equation and when is it a bad idea?
$\beta = (X^T X)^{-1} X^T y$. It is slow for a very large number of features (matrix inversion is $O(p^3)$) and fails if $X^T X$ is singular (perfect multicollinearity).

B. Questions about the graphs

G1. What does the Actual vs Predicted plot show?
Each dot is one test sample. The red dashed line is perfect prediction (y = x). Points near it are accurate; wide scatter means large errors.

G2. Why is the cloud flatter than the red line?
The model shrinks predictions toward the mean: it over-predicts low values and under-predicts high values. This is common when R^2 is moderate.

G3. What does the residual plot show and what should a good one look like?
Residual = actual minus predicted. A good plot is a random cloud around zero with constant spread and no pattern. A curve means non-linearity; a funnel shape means heteroscedasticity. Either means the linear model is not ideal.

G4. What does the residual histogram and Q-Q plot tell you?
Whether residuals are approximately normal. A bell shape centred at zero and dots on the Q-Q line support the normality assumption; heavy tails show a few large errors.

G5. What does the correlation heatmap tell you?
The pairwise correlation between variables (-1 to +1). A strong correlation between a feature and the target means a good predictor. A strong correlation between two features signals multicollinearity.

G6. What does the single-feature regression plot show?
A scatter of one feature versus the target with the best-fit line. The slope indicates the direction and strength of the relationship.

G7. What does the coefficient bar chart show?
The size and sign of each coefficient: red bars increase the target, blue bars decrease it. Because features are on the same scale here, bar length compares importance (but unstable if features are collinear).

G8. What does the VIF bar chart show?
How inflated each coefficient's variance is because of collinearity. Bars above the red line (5) indicate problematic features.

C. Code questions

C1. What do `fit()` and `predict()` do?
`fit` learns the coefficients from training data; `predict` computes $\hat{y}$ for new data using them.

C2. Why do we evaluate on the test set and not the training set?
Training error is optimistic; the test set shows performance on unseen data.

C3. What does `random_state=42` do?
Makes the random split reproducible.

C4. What does `model.coef_` and `model.intercept_` return?
Coefficients (one per feature) and the intercept b_0.

C5. Why `np.sqrt(mse)` for RMSE?
RMSE is defined as the square root of MSE, which returns the error to the target's units.

C6. What does `cross_val_score(..., cv=5)` do?
It splits the data into 5 folds, trains on 4 and tests on 1, repeats 5 times, and returns the 5 scores.

Practical 4: Polynomial Regression and the Bias-Variance Trade-off

A. Concept questions

Q1. What is polynomial regression?
Regression where the relationship between x and y is modelled as an n-th degree polynomial: $y = b_0 + b_1x + b_2x^2 + \cdots + b_dx^d$. It captures curved relationships.

Q2. Why is it called a linear model even though the curve is non-linear?
Because it is linear in the parameters $b_0, b_1, \ldots$. We transform x into x, x^2, x^3 and fit an ordinary linear regression on these new features.

Q3. What does `PolynomialFeatures(degree=2)` do?
For one input x it creates $[1, x, x^2]$. For two inputs (a, b) it creates $[1, a, b, a^2, ab, b^2]$, including interaction terms. `include_bias=False` drops the constant column because the regression already has an intercept.

Q4. When do you use polynomial regression?
When a scatter plot shows a curved relationship that a straight line cannot capture.

Q5. What is bias?
Error from wrong or overly simple assumptions in the model. High bias causes underfitting.

Q6. What is variance?
Error from sensitivity to small changes in the training data. High variance causes overfitting because the model learns noise.

Q7. Explain the bias-variance trade-off.
Increasing model complexity lowers bias but raises variance. Expected test error = Bias² + Variance + Irreducible noise. The goal is the sweet spot of minimum total error.

Q8. What is underfitting?
The model is too simple to capture the pattern. Both training and test errors are high.

Q9. What is overfitting?
The model fits training data (and its noise) too closely. Training error is very low but test error is high.

Q10. How do you detect overfitting?
A large gap between training performance and test performance. Learning curves and cross-validation also show it.

Q11. How do you reduce overfitting?
Lower the polynomial degree, get more data, use regularisation (Ridge/Lasso), use cross-validation for model selection, remove features, or use early stopping.

Q12. How do you choose the best degree?
Compare test or cross-validation error for several degrees and choose the lowest, preferring the simplest degree when scores are similar.

Q13. What is irreducible error?
Noise inherent in the data that no model can remove.

Q14. What happens to the number of features as the degree grows?
It grows very quickly with multiple inputs (combinatorially), which raises computation and the risk of overfitting.

Q15. Why scale features before polynomial regression?
x^{12} creates huge values and numerical instability. Scaling keeps the numbers manageable.

Q16. Can training R^2 alone judge the model?
No. Training R^2 always rises with degree. Judge on test data or cross-validation.

Q17. Can polynomial regression extrapolate well?
No. Outside the training range, high-degree polynomials give extreme, unreliable predictions.

Q18. What is Runge's phenomenon?
High-degree polynomials oscillate wildly near the edges of the data range, which is why the high-degree curve swings at the ends.

Q19. What is the difference between polynomial regression and linear regression?
Linear regression fits a straight line (or hyperplane). Polynomial regression fits a curve by adding powers of the inputs as extra features. Linear regression is polynomial regression of degree 1.

Q20. Is polynomial regression a parametric model?
Yes; the number of parameters is fixed by the degree.

Q21. Which metrics did you use?
MSE and R^2 on both train and test sets, plus cross-validated MSE.

B. Questions about the graphs

G1. What does the scatter plot of horsepower vs MPG show?
A negative, curved relationship: MPG decreases as horsepower increases, steeply at first and then flattening. This is why a straight line is not enough.

G2. What does the degree 1 curve look like and what does it mean?
A straight line that misses the curvature. Both train and test errors are high, so it underfits (high bias, low variance).

G3. What does a very high degree curve look like?
A wiggly curve that bends to follow individual points and can swing wildly at the edges. Train error is very low but test error is not better, so it overfits (low bias, high variance).

G4. Explain the train/test error versus degree graph.
Train error keeps falling as degree rises. Test error falls first, reaches a minimum (the best degree) and then flattens or rises. The gap between the two lines widens with overfitting.

G5. Where is the best model on that graph?
At the degree with the lowest test (or cross-validation) error, where bias and variance are balanced. In my run the test error is lowest at degree 6 and the CV error at degree 7, but degree 2 is almost as good, so I prefer the simpler model.

G5b. Your test-error curve is almost flat after degree 2. Why is there no clear U-shape?
The dataset has one input feature and about 270 training points, so even a high-degree polynomial is well constrained in the dense region and the test split changes little. The overfitting shows up where data is sparse (the edges), which is why cross-validation error rises and the bootstrap variance explodes at high degrees. With less data or more noise the U-shape on the test curve would be much sharper.

G6. Why does the curve swing at the ends for high degrees?
High-degree terms grow very fast outside the dense data region, so small coefficient changes cause large swings (Runge's phenomenon). Few data points at the edges also give the model little to constrain it.

G7. What does the bias-variance graph show?
Bias² decreases and variance increases as the degree rises; the total error is lowest where they balance.

G8. What do the error bars in the cross-validation plot mean?
The standard deviation of the error across the 5 folds. Large bars mean the score is unstable and the differences between nearby degrees may not be meaningful.

G9. What does the Ridge plot show compared with the plain degree-12 plot?
The Ridge curve is smoother and less wiggly because the penalty shrinks large coefficients, so variance decreases. In my run the test MSE was slightly higher than the unregularised model (about 20.7 vs 19.7) because the penalty added a little bias and the unregularised model was not badly overfitted on this data. Ridge helps most when overfitting is severe.

C. Code questions

C1. Why use a pipeline?
It chains scaling, polynomial feature creation and regression so the same steps are applied consistently to train and test data and leakage is avoided.

C2. What does `make_pipeline` do?
Builds a Pipeline with automatically named steps.

C3. Why `np.linspace` and `reshape(-1, 1)`?
To create a smooth grid of x values (one column) for drawing the fitted curve, since scikit-learn expects a 2-D array.

C4. Why do we reshape X to 2-D?
scikit-learn features must be shaped (n_samples, n_features).

C5. What does the bootstrap do in Step 7?
It resamples the training data with replacement many times, fits a model on each resample, and uses the spread of the predictions to estimate variance.

Practical 5: Regularisation with Ridge and Lasso Regression

A. Concept questions

Q1. What is regularisation and why use it?
Adding a penalty on the size of the coefficients to the loss function. It prevents overfitting, stabilises models with multicollinearity and (Lasso) performs feature selection.

Q2. What is Ridge regression (L2)?
Linear regression with a penalty of alpha times the sum of squared coefficients: $\min \sum (y_i - \hat{y}_i)^2 + \alpha \sum \beta_j^2$. It shrinks coefficients but does not make them exactly zero.

Q3. What is Lasso regression (L1)?
Linear regression with a penalty of alpha times the sum of absolute coefficients: $\min \sum (y_i - \hat{y}_i)^2 + \alpha \sum |\beta_j|$. It can shrink some coefficients exactly to zero, so it also selects features.

Q4. Difference between Ridge and Lasso?
Ridge uses L2 and keeps all features with smaller weights; best when all features matter and are correlated. Lasso uses L1 and removes features; best when many features are irrelevant (sparse models).

Q5. Why does Lasso give exact zeros but Ridge does not?
The L1 constraint region is a diamond with corners on the axes, so the loss contours often touch at a corner where a coefficient is zero. The L2 region is a circle with no corners, so exact zeros are rare.

Q6. What does alpha (lambda) control?
The strength of the penalty. Alpha = 0 gives ordinary linear regression. Larger alpha means stronger shrinkage, higher bias and lower variance. Too large leads to underfitting.

Q7. How do you choose alpha?
By cross-validation (`RidgeCV`, `LassoCV`, `GridSearchCV`), choosing the alpha with the lowest validation error.

Q8. What is multicollinearity?
High correlation between independent variables, which makes coefficient estimates unreliable (large, unstable, sign-flipping). Detect it with the correlation matrix or VIF.

Q9. How does Ridge handle multicollinearity?
With correlated features, OLS coefficients become huge and unstable. The L2 penalty spreads weight across correlated features and stabilises the solution; it also makes $X^T X + \alpha I$ always invertible.

Q10. How does Lasso behave with correlated features?
It tends to pick one of them and set the others to zero, and the choice can be arbitrary and change between samples.

Q11. What is Elastic Net?
A combination of L1 and L2 penalties, useful when features are correlated and you also want sparsity.

Q12. Why scale data before Ridge or Lasso?
The penalty treats all coefficients equally, so features with large scales would be penalised unfairly. Standardising makes the penalty fair.

Q13. Is the intercept penalised?
No, only the feature coefficients are.

Q14. Effect of regularisation on bias and variance?
It increases bias slightly and decreases variance, which usually lowers test error when the model was overfitting.

Q15. When would you not use regularisation?
When the model already underfits, or when there are few features and plenty of data.

Q16. Is Lasso a feature selection method?
Yes, an embedded one, because selection happens during training.

Q17. What is overfitting and how did you see it in this practical?
When a model fits training data too closely and performs poorly on new data. In Part B, OLS had a high training R^2 but a low test R^2.

Q18. What happens when alpha = 0? When alpha is very large?
Alpha = 0 is plain OLS. A very large alpha pushes all coefficients to (nearly) zero, so the model predicts roughly the mean (underfitting).

Q19. Does regularisation always improve test accuracy?
No. If the model is not overfitting (as in Part A on 10 features), the improvement is small or none. It helps most with many features, few samples or collinearity.

Q20. What is the VIF?
Variance Inflation Factor: $1/(1-R_j^2)$, where R_j^2 comes from regressing feature j on the others. Values above 5 to 10 indicate problematic collinearity.

Q21. Closed-form solution of Ridge?
$\beta = (X^T X + \alpha I)^{-1} X^T y$. Lasso has no closed form and is solved with coordinate descent.

Q22. Does Ridge perform feature selection?
No. It keeps all features.

B. Questions about the graphs

G1. Explain the coefficient bar plot.
Each group of bars is one feature; the colours are the models. OLS has the largest coefficients (large with opposite signs for correlated features `s1` and `s2`). Ridge bars are smaller. Lasso has some bars exactly at zero, so those features are removed.

G2. Explain the Ridge coefficient path.
X-axis: alpha (log). Y-axis: coefficient values. At small alpha the coefficients equal the OLS values. As alpha grows, all coefficients shrink smoothly toward zero but never exactly reach it.

G3. Explain the Lasso coefficient path.
As alpha grows, coefficients hit exactly zero one at a time and stay there. The last features to reach zero are the most important ones.

G4. What does the validation curve (error vs alpha) show?
Train and test error versus alpha. Tiny alpha gives overfitting (low train error, high test error); huge alpha gives underfitting (both high); the minimum of the test curve is the best alpha.

G5. What does the gap between train and test R^2 show?
A large gap means overfitting. Regularisation shrinks the gap.

G6. What does the bootstrap (s1 vs s2) scatter show?
How much the two coefficients vary across resampled datasets. A wide tilted cloud for OLS means unstable, negatively tied estimates (multicollinearity). A tighter cloud for Ridge means regularisation stabilised them.

G7. What does the "non-zero coefficients vs alpha" plot show?
The number of features Lasso keeps decreases as alpha increases, until none remain.

G8. What does the correlation heatmap show?
Which features are strongly correlated (for example `s1` and `s2` at about 0.9), indicating multicollinearity.

C. Code questions

C1. What is the difference between `Ridge(alpha=10)` and `RidgeCV`?
`Ridge` uses a fixed alpha. `RidgeCV` tries many alphas by cross-validation and keeps the best.

C2. Why `max_iter` in Lasso?
Lasso is solved iteratively (coordinate descent); more iterations may be needed to converge.

C3. Why `np.logspace` for alphas?
The effect of alpha changes over orders of magnitude, so a logarithmic grid covers the range efficiently.

C4. Why scale inside a pipeline in Part B?
So scaling is fitted only on the training data and applied automatically to test data.

C5. How do you get the features Lasso kept?
Those where `model.coef_` is not zero.

Practical 6: Logistic Regression, Confusion Matrix, Precision, Recall and F1-score

A. Concept questions

Q1. What is logistic regression? Is it regression or classification?
Despite the name, it is a classification algorithm. It models the probability that an input belongs to class 1 using the sigmoid function and then applies a threshold (default 0.5).

Q2. Why not use linear regression for classification?
Linear regression outputs values outside 0 to 1, is sensitive to outliers, and its squared-error loss is not suited to categories. Logistic regression gives valid probabilities.

Q3. What is the sigmoid function?
$\sigma(z) = 1/(1 + e^{-z})$. It squashes any real number into (0, 1), with $\sigma(0) = 0.5$.

Q4. What loss function does logistic regression use?
Log loss (binary cross-entropy): $-[y \log p + (1 - y) \log(1 - p)]$, minimised with gradient descent or solvers such as lbfgs.

Q5. What is the decision boundary?
The surface where the predicted probability equals the threshold (0.5), i.e., where $z = 0$. It is linear in logistic regression.

Q6. What are odds and log-odds?
Odds = $p/(1 - p)$. Log-odds (logit) = $\log(p/(1 - p)) = z$, a linear combination of the features. Each coefficient is the change in log-odds for a one-unit change in that feature; e^b is the odds ratio.

Q7. What is a confusion matrix?
A table comparing actual and predicted classes: TP, TN, FP and FN counts.

Q8. Define TP, TN, FP, FN.
TP: predicted positive and actually positive. TN: predicted negative and actually negative. FP: predicted positive but actually negative (false alarm, Type I). FN: predicted negative but actually positive (missed case, Type II).

Q9. Define accuracy, precision, recall and F1.
Accuracy: share of all predictions that are correct. Precision: of predicted positives, how many are truly positive. Recall (sensitivity): of actual positives, how many were found. F1: harmonic mean of precision and recall.

Q10. Why is accuracy not enough?
With imbalanced classes a model that always predicts the majority class can have high accuracy but be useless. Precision, recall and F1 expose this.

Q11. When do you prefer high precision and when high recall?
High recall when missing a positive is costly (cancer screening, fraud). High precision when a false alarm is costly (spam filter blocking real mail, legal decisions).

Q12. Why is F1 the harmonic mean and not the arithmetic mean?
The harmonic mean is dominated by the lower value, so a model cannot get a high F1 by being excellent at one metric and terrible at the other.

Q13. How does changing the threshold affect results?
Lower threshold: more positives, recall up, precision down. Higher threshold: precision up, recall down.

Q14. What is specificity?
$TN/(TN+FP)$: the share of actual negatives correctly identified (true negative rate).

Q15. What is the ROC curve and AUC?
ROC plots TPR against FPR at all thresholds. AUC is the area under it: 1.0 is perfect, 0.5 is random. It equals the probability that a random positive is ranked above a random negative.

Q16. ROC curve vs precision-recall curve: when to use which?
Use PR curves for heavily imbalanced data because ROC can look optimistic when negatives dominate.

Q17. What is the role of the regularisation parameter C?
C is the inverse of regularisation strength. Small C means stronger regularisation (simpler model, less overfitting); large C means weaker regularisation. By default scikit-learn applies L2 regularisation.

Q18. Can logistic regression do multiclass classification?
Yes, using One-vs-Rest or the softmax (multinomial) approach.

Q19. What are the assumptions of logistic regression?
Binary (or categorical) target, independent observations, little multicollinearity, a linear relationship between features and the log-odds, and a reasonably large sample size.

Q20. Why is `stratify=y` used?
It keeps the same class proportions in the train and test sets, which matters for imbalanced data.

Q21. Why did you scale the features?
Logistic regression is solved by gradient-based optimisation and regularised; scaling speeds up convergence and makes coefficients comparable.

Q22. What is Type I and Type II error?
Type I = false positive (false alarm). Type II = false negative (missed detection).

Q23. What are macro, micro and weighted averages?
Macro: simple mean of per-class scores. Weighted: mean weighted by class support. Micro: computed globally from total TP, FP, FN (equals accuracy for single-label multiclass).

Q24. Is logistic regression affected by outliers?
Less than linear regression, but extreme values can still distort the fit; scaling and outlier handling help.

Q25. Why did you treat malignant as the positive class?
In screening, the event we want to detect is the disease. Treating it as positive means recall measures how many cancers we catch and FN is the dangerous error. (In scikit-learn's raw labels, 1 is benign, so the labels were flipped.)

B. Questions about the graphs

G1. How do you read the confusion matrix heatmap?
Rows are actual classes and columns are predicted classes. The diagonal shows correct predictions; off-diagonal cells are mistakes. A dark diagonal and light off-diagonal means a good model.

G2. Which cell is the most dangerous here?
False Negative (bottom-left): a malignant tumour predicted as benign, so the patient is missed.

G3. What does the normalised confusion matrix show?
Percentages within each actual class. The bottom-right cell is recall and the top-left is specificity.

G4. What does the ROC curve show?
The trade-off between true positive rate (y) and false positive rate (x) at all thresholds. A curve near the top-left is best; the diagonal is random guessing.

G5. What is AUC?
Area Under the ROC Curve. 1.0 is perfect, 0.5 is random.

G6. What does the sigmoid curve look like?
An S-shape from 0 to 1 crossing 0.5 at z = 0. It turns any real number into a probability.

G7. What does the precision/recall/F1 vs threshold plot show?
As the threshold rises, precision generally increases and recall decreases; F1 peaks where the two are balanced. It helps choose a threshold for the application.

G8. What does the probability histogram tell you?
How confident the model is. Well-separated peaks near 0 and 1 mean confident, accurate predictions; bars near 0.5 are uncertain cases and the likely errors.

G9. What does the coefficient bar chart show?
The influence of each feature on the log-odds of malignancy: positive (red) increases risk, negative (blue) decreases it; larger bars matter more.

G10. What does the class distribution bar chart tell you?
Whether the classes are balanced. Here the data is mildly imbalanced (about 63% benign), so accuracy alone is not enough.

C. Code questions

C1. What is the difference between `predict` and `predict_proba`?
`predict` returns class labels (using the 0.5 threshold). `predict_proba` returns the probability of each class.

C2. Why `predict_proba(X)[:, 1]`?
It selects the column with the probability of the positive class (class 1).

C3. What does `confusion_matrix(y_true, y_pred).ravel()` return?
The four counts in the order TN, FP, FN, TP (for labels 0 and 1).

C4. Why `max_iter=1000`?
The solver may need more iterations than the default (100) to converge.

C5. Why is the scaler fitted only on training data?
To avoid data leakage from the test set.

C6. What does `zero_division=0` do?
Returns 0 instead of a warning/error when a precision denominator is zero (no predicted positives).

Practical 7: Support Vector Machines with Linear and Non-linear Kernels

A. Concept questions

Q1. What is an SVM?
A supervised algorithm that finds the hyperplane separating the classes with the **maximum margin**. It is used for classification (SVC) and regression (SVR).

Q2. What is a hyperplane?
A decision surface of dimension n-1 in n-dimensional space: a line in 2D, a plane in 3D.

Q3. What are support vectors?
The training points closest to the hyperplane (on or inside the margin). They alone determine its position; removing other points does not change it.

Q4. What is the margin and why maximise it?
The distance between the hyperplane and the nearest points of each class. A larger margin gives better generalisation and lower risk of overfitting.

Q5. Hard margin vs soft margin?
Hard margin allows no misclassification and needs perfectly separable data. Soft margin allows some violations, controlled by C, so it works with noisy, overlapping data.

Q6. What does C control?
The trade-off between a wide margin and training errors. Large C: fewer training errors, narrow margin, overfitting risk. Small C: wide margin, more errors allowed, underfitting risk.

Q7. What is the kernel trick?
Computing the dot product of data in a higher-dimensional space directly with a kernel function, without actually transforming the data. It lets SVM learn non-linear boundaries cheaply.

Q8. Name the common kernels and when to use them.
Linear: $K = x^T z$, for linearly separable or high-dimensional sparse data (text). Polynomial: $K = (\gamma x^T z + r)^d$, for curved boundaries. RBF (Gaussian): $K = \exp(-\gamma \|x - z\|^2)$, the default for general non-linear data. Sigmoid: $K = \tanh(\gamma x^T z + r)$, rarely used.

Q9. What is gamma in the RBF kernel?
It controls how far a single training point's influence reaches. High gamma: small influence, complex boundary, overfitting. Low gamma: large influence, smooth boundary, underfitting.

Q10. Why must features be scaled for SVM?
SVM uses distances, so large-range features would dominate the margin and kernel calculation.

Q11. How does SVM do multi-class classification?
By combining binary classifiers: one-vs-one (scikit-learn's SVC) or one-vs-rest.

Q12. Advantages and disadvantages of SVM?
Advantages: effective in high dimensions, robust when a clear margin exists, memory-efficient (uses only support vectors), flexible via kernels. Disadvantages: slow on very large datasets, needs scaling and tuning, no direct probabilities (`probability=True` adds cost), and performs poorly with heavy noise or overlapping classes.

Q13. Linear SVM vs logistic regression?
Both are linear classifiers. SVM maximises the margin using hinge loss and depends only on support vectors; logistic regression uses log loss, gives probabilities and uses all points.

Q14. What is hinge loss?
$\max(0, 1 - yf(x))$: zero for points correctly classified beyond the margin, growing linearly for violations.

Q15. How do you choose the kernel and parameters?
Start with linear and RBF, then tune C and gamma with GridSearchCV and cross-validation.

Q16. Why did linear and RBF give similar accuracy on Breast Cancer?
The data is almost linearly separable, so a complex kernel adds no benefit. Non-linear kernels help only when the classes are not linearly separable.

Q16b. Which kernel did best and worst in your run, and why?
RBF was best (0.972) with linear close behind (0.958); polynomial was worst (0.895, recall 0.72). The default degree-3 polynomial kernel is sensitive to scaling and its settings (`degree`, `coef0`), so it needs tuning. The linear-vs-RBF gap is only 2 patients on 143 test samples, so it is not significant.

Q17. What is the difference between `SVC` and `LinearSVC`?
`SVC(kernel='linear')` uses libsvm (supports kernels, slower on big data). `LinearSVC` uses liblinear (linear only, much faster on large datasets).

Q18. How do you get class probabilities from an SVM?
Set `probability=True`; scikit-learn fits Platt scaling using internal cross-validation (slower).

Q19. What does the "dual form" of SVM mean?
The optimisation is written in terms of Lagrange multipliers, where only support vectors have non-zero multipliers, and the data appears only through dot products, which is what makes the kernel trick possible.

Q20. Is SVM sensitive to outliers?
A hard-margin SVM is; a soft-margin SVM with smaller C is more robust.

Q21. What is `gamma='scale'`?
The default $\gamma = 1/(n_{features} \cdot Var(X))$, which adapts to the data scale.

B. Questions about the graphs

G1. What does the decision boundary plot show?
Coloured regions show the class predicted at each point in the plane; the boundary between colours is the decision boundary. Points are the data coloured by true class; points in the wrong region are misclassified.

G2. How does the linear boundary differ from the RBF boundary?
Linear gives straight lines (hyperplanes); RBF gives smooth, curved, flexible regions that wrap around clusters.

G3. What are the circled points?
Support vectors.

G4. Why does the linear kernel fail on the moons data?
The classes are interleaved and not linearly separable, so no straight line can separate them. RBF bends the boundary around the moons.

G5. What would a very high gamma look like?
Tiny islands around individual points (very wiggly boundary): overfitting. A very low gamma gives an almost straight boundary: underfitting.

G6. What if C is very large?
The model tries to classify every training point correctly: narrow margin and possible overfitting. Small C gives a wider margin that allows some errors (more generalisation).

G7. How do you read the 3x3 C/gamma grid?
Moving down increases C, moving right increases gamma. Top-left is smooth and underfit; bottom-right is wiggly and overfit; the middle is the best balance.

G8. How do you read the GridSearch heatmap?
Each cell is the cross-validated accuracy for a (C, gamma) pair; the highest value gives the best hyperparameters.

G9. What does the support-vector count bar chart show?
How many training points define the boundary for each kernel. Fewer means a simpler boundary.

G10. What do the confusion matrices show?
Correct (diagonal) and wrong (off-diagonal) predictions for each kernel; FN means a missed malignant case.

C. Code questions

C1. Why scale before SVM, and why fit the scaler on training data only?
SVM is distance-based; fitting on train only avoids data leakage.

C2. What does `n_support_` return?
The number of support vectors for each class.

C3. What does `GridSearchCV` do?
Tries every combination in the parameter grid with cross-validation and keeps the best.

C4. What does `np.meshgrid` do in the boundary plot?
Creates a grid of points covering the plane; the model predicts a class for each, and `contourf` colours the regions.

Practical 8: K-Means Clustering (Elbow Method and Silhouette Score)

A. Concept questions

Q1. What is clustering?
Grouping similar data points together without using labels (unsupervised learning).

Q2. How does the K-Means algorithm work?
(1) Choose k. (2) Initialise k centroids. (3) Assign every point to its nearest centroid. (4) Recompute each centroid as the mean of its points. (5) Repeat steps 3 and 4 until assignments stop changing (or max iterations).

Q3. What is inertia / WCSS?
Within-Cluster Sum of Squares: the total of squared distances between each point and its cluster centroid. Smaller means tighter clusters.

Q4. What is the Elbow method?
Plot inertia for different k and choose the k at the bend of the curve, after which improvement is marginal.

Q5. What is the silhouette score?
A measure of how similar a point is to its own cluster compared with other clusters, from -1 to +1. The average over all points rates the clustering.

Q6. Elbow vs silhouette?
Elbow is visual and subjective, and only looks at compactness. Silhouette considers compactness and separation and gives a numeric score to maximise.

Q7. Why did the elbow and silhouette methods disagree on Iris?
Versicolor and virginica overlap, so geometrically the data looks like 2 groups (silhouette prefers k = 2) although there are 3 species (elbow suggests 3). The methods measure different things; combine them with domain knowledge.

Q8. Why is scaling important for K-Means?
It uses Euclidean distance, so features with larger ranges dominate.

Q9. What is K-Means++ initialisation?
A smarter way to choose starting centroids, spreading them far apart. It gives faster convergence and better results than random initialisation (the default in scikit-learn).

Q10. What is `n_init`?
The number of times K-Means is run with different starting centroids; the run with the lowest inertia is kept.

Q11. Limitations of K-Means?
You must choose k in advance; sensitive to initialisation and outliers; assumes spherical, similarly-sized clusters; fails on non-convex shapes (moons, rings); numeric data only.

Q12. Does K-Means always find the global optimum?
No. It converges to a local optimum, so we run it several times.

Q13. Time complexity?
$O(n \cdot k \cdot i \cdot d)$ (n points, k clusters, i iterations, d features). It scales well to large data.

Q14. Difference between K-Means and KNN?
K-Means is unsupervised clustering and k is the number of clusters. KNN is a supervised classifier and k is the number of neighbours.

Q15. How do you evaluate clustering without labels?
Silhouette score, Davies-Bouldin index (lower is better), Calinski-Harabasz index (higher is better), inertia. With labels: Adjusted Rand Index.

Q16. K-Means vs K-Medoids?
K-Medoids uses actual data points as centres and is more robust to outliers; K-Means uses means.

Q17. Hard vs soft clustering?
K-Means is hard clustering (each point belongs to exactly one cluster). Gaussian Mixture Models do soft clustering (probabilities).

Q18. What is a centroid?
The mean position of all points in a cluster.

Q19. What is the objective function K-Means minimises?
The within-cluster sum of squared distances (inertia).

Q20. Why does inertia always decrease as k increases?
More centroids means every point is closer to some centroid. With k = n, inertia is zero, so we cannot just pick the minimum; we look for the elbow.

Q21. What is the Adjusted Rand Index?
A measure of agreement between two clusterings (for example clusters vs true labels), adjusted for chance: 1 is perfect, around 0 is random.

Q22. What if the elbow is not clear?
Use the silhouette score, the gap statistic, other indices (Davies-Bouldin), or domain knowledge.

B. Questions about the graphs

G1. How do you read the Elbow curve?
X-axis is k, Y-axis is inertia. Inertia always falls as k grows; the optimal k is at the elbow, where the drop becomes slow and the curve flattens.

G2. How do you read the silhouette plot (score vs k)?
Choose the k with the highest average silhouette. Values near +1 mean tight, well-separated clusters; near 0 means overlapping; negative means points are probably in the wrong cluster.

G3. How do you read a silhouette diagram (knife plot)?
Each knife is a cluster; each line is a sample sorted by silhouette value. The red dashed line is the average. Knives of similar width that extend past the red line are good; a thin knife or negative parts mean a poor cluster.

G4. What do the red X marks in the scatter plot mean?
The centroids (the mean position of each cluster). Each point belongs to its nearest centroid.

G5. Why are the clusters plotted using PCA?
The data has 4 features, which cannot be drawn directly; PCA projects it to 2D for visualisation.

G6. What does the crosstab (clusters vs species) tell you?
How well the clusters match the true labels: setosa is perfectly isolated, versicolor and virginica are partly mixed.

G7. What does the inertia-per-iteration plot show?
Convergence: inertia decreases at every iteration and stops changing when the algorithm has converged.

G8. What does the blobs example show?
When clusters are well separated, the elbow is sharp and the silhouette peak agrees with it (k = 5).

C. Code questions

C1. What does `km.labels_` and `km.cluster_centers_` contain?
The cluster number of every sample, and the coordinates of the centroids.

C2. Why `random_state=42`?
K-Means initialisation is random; the seed makes results reproducible.

C3. Why are cluster numbers arbitrary?
Clusters are just labels 0..k-1 assigned by the algorithm; they have no inherent meaning or order.

C4. Why PCA after K-Means rather than before?
We cluster in the full 4-D space to keep all information and use PCA only to draw the result.

Practical 9: Density-Based Clustering with DBSCAN

A. Concept questions

Q1. What is DBSCAN?
Density-Based Spatial Clustering of Applications with Noise. It groups points that are closely packed (high density) and marks points in low-density regions as noise.

Q2. What are its two parameters?
`eps`, the neighbourhood radius, and `min_samples`, the minimum number of points to form a dense region.

Q3. Explain core, border and noise points.
Core: at least `min_samples` points within eps. Border: within eps of a core point but not core itself. Noise: neither.

Q4. Explain the algorithm.
Pick an unvisited point. If it is a core point, start a new cluster and add all density-reachable points, expanding through other core points. If it is not core, mark it as noise (it may later turn out to be a border point). Repeat until all points are visited.

Q5. What is density-reachable and density-connected?
A point q is density-reachable from p if there is a chain of core points from p to q, each within eps of the next. Two points are density-connected if both are reachable from a common core point.

Q6. DBSCAN vs K-Means?
DBSCAN needs no number of clusters, finds arbitrary shapes, detects noise, but is sensitive to eps and struggles with varying densities. K-Means needs k, finds only spherical clusters, assigns every point to a cluster (no noise concept) and is faster on large data.

Q7. How do you choose eps and min_samples?
`min_samples` is about 2 × the number of dimensions (at least 3 to 5). eps comes from the knee of the k-distance plot.

Q8. Is DBSCAN sensitive to scaling?
Yes, it is distance-based, so scale the features first.

Q9. What are the drawbacks of DBSCAN?
Hard to choose eps; fails when clusters have very different densities (one eps cannot fit all); poor in high dimensions (distances lose meaning); border points can be assigned differently depending on the processing order.

Q10. What does label -1 mean?
Noise (an outlier).

Q11. Can DBSCAN be used for anomaly detection?
Yes, the noise points are candidate anomalies.

Q12. What distance metric does it use?
Euclidean by default; others (Manhattan, cosine) can be set with `metric=`.

Q13. Time complexity?
$O(n \log n)$ with a spatial index such as a KD-tree, $O(n^2)$ without.

Q14. What is HDBSCAN?
An extension that removes the need for eps and handles clusters of varying density.

Q15. How do you evaluate DBSCAN?
Silhouette score on non-noise points, Davies-Bouldin index, or Adjusted Rand Index if true labels exist; also report the noise percentage.

Q16. Does DBSCAN need the number of clusters in advance?
No. The number of clusters is a result.

Q17. Is DBSCAN deterministic?
Core points and noise are deterministic; border points reachable from two clusters may be assigned to either, depending on order.

Q18. What happens if eps is too small? Too large?
Too small: most points become noise and many tiny clusters appear. Too large: clusters merge into one.

Q19. What happens if `min_samples` is increased?
A denser region is required, so more points become noise and small clusters disappear; the result is more conservative.

Q20. Why did DBSCAN not separate versicolor and virginica in Iris?
They overlap and form one continuous dense region, so there is no low-density gap for DBSCAN to cut along.

Q21. Can DBSCAN find clusters of different shapes?
Yes, any shape (even non-convex ones like moons, rings and spirals).

B. Questions about the graphs

G1. How do you read the k-distance plot?
Each point's distance to its k-th nearest neighbour is sorted ascending. The curve is flat for dense-region points and rises sharply for sparse points and outliers. The y-value at the knee is the best eps.

G2. What do the black crosses mean in the cluster plot?
Noise points (label -1): points in low-density regions that belong to no cluster.

G3. Why does DBSCAN beat K-Means on the moons?
DBSCAN follows dense regions and forms arbitrarily shaped clusters; K-Means assumes round clusters around centroids, so it splits the moons with a straight boundary.

G4. How many clusters did DBSCAN find and how do you know?
Count the unique labels excluding -1. The number of clusters comes from the data, not from the user.

G5. What does the eps sensitivity grid show?
Tiny eps gives mostly noise and many clusters; the right eps gives the true structure; a large eps merges everything into one cluster with no noise.

G6. What do the core, border and noise colours mean?
Core (blue): dense interior points. Border (yellow): on the edge of a cluster. Noise (black x): outliers.

G7. What does the Iris result tell you about DBSCAN?
It finds setosa as one cluster and merges versicolor and virginica, because density does not separate them.

C. Code questions

C1. What does `db.labels_` contain?
The cluster index of each point; -1 means noise.

C2. What does `db.core_sample_indices_` contain?
Indices of the core points.

C3. Why add random outliers in Part A?
To have genuine noise points to detect, which demonstrates DBSCAN's outlier handling.

C4. Why `NearestNeighbors(n_neighbors=min_samples)`?
To get each point's distance to its min_samples-th neighbour for the k-distance plot.

Practical 10: Ensemble Learning (Random Forest and Boosting) vs a Baseline Model

A. Concept questions

Q1. What is ensemble learning?
Combining multiple models (weak learners) to get a better, more stable model than any single one.

Q2. Types of ensemble methods?
Bagging (Random Forest), Boosting (AdaBoost, Gradient Boosting, XGBoost), Stacking and Voting.

Q3. What is bagging?
Bootstrap Aggregating: train many models on random samples drawn with replacement, then average (regression) or vote (classification). It reduces **variance**.

Q4. What is Random Forest?
An ensemble of decision trees built with bagging **plus random feature selection at each split**, which de-correlates the trees. The final output is the majority vote.

Q5. Why does Random Forest beat a single tree?
A single deep tree overfits (high variance). Averaging many different trees cancels individual errors and reduces variance.

Q6. What is a bootstrap sample and OOB error?
A bootstrap sample is drawn with replacement from the training data (about 63% unique rows). The remaining ~37% (out-of-bag) rows give a free error estimate (`oob_score=True`).

Q7. What is boosting?
Sequentially training weak learners where each tries to correct the errors of the previous ones, giving more weight to misclassified examples. It mainly reduces **bias**.

Q8. AdaBoost vs Gradient Boosting?
AdaBoost re-weights misclassified samples and combines learners with weights. Gradient Boosting fits each new tree to the residual errors (the negative gradient of the loss).

Q9. Key hyperparameters of Random Forest?
`n_estimators`, `max_depth`, `max_features`, `min_samples_split`, `min_samples_leaf`.

Q10. Key hyperparameters of boosting?
`n_estimators`, `learning_rate` (smaller rate needs more trees), `max_depth`, `subsample`.

Q11. Which one is more prone to overfitting?
Boosting, especially with many rounds and a high learning rate. Random Forest is more robust.

Q12. What is a weak learner?
A model only slightly better than random guessing, such as a shallow tree (stump).

Q13. Is scaling required for tree ensembles?
No, trees split on thresholds and are scale-invariant.

Q14. How is feature importance calculated?
By the total decrease in impurity (Gini or entropy) contributed by splits on that feature, averaged over all trees.

Q15. What is a baseline model and why use it?
A simple model used as a reference. If a complex model cannot beat it, the added complexity is not justified. The Dummy model is the lowest bar; the single decision tree is the practical baseline.

Q16. What are the disadvantages of ensembles?
Less interpretable, higher computation and memory, longer training time.

Q17. What is a decision tree and what is Gini impurity?
A tree-shaped model that splits data on feature thresholds. Gini = $1 - \sum p_i^2$; it is 0 for a pure node, and the best split reduces it the most.

Q18. What is stacking?
Training a meta-model on the predictions of several different base models.

Q19. Bagging vs boosting?
Bagging: parallel, independent trees on bootstrap samples, reduces variance. Boosting: sequential trees, each fixing the previous errors, reduces bias.

Q20. Why does a single decision tree reach 100% training accuracy?
An unrestricted tree keeps splitting until every leaf is pure, memorising the training data: overfitting.

Q21. Why is `max_features` important in Random Forest?
Using only a random subset of features at each split forces trees to be different (de-correlated), which makes averaging more effective.

Q22. What is XGBoost?
An optimised, regularised implementation of gradient boosting that is fast and widely used.

Q23. How does a Random Forest do regression?
Each tree predicts a number, and the forest averages them.

Q24. Can feature importance be misleading?
Yes: it is biased toward features with many unique values, splits importance across correlated features, and gives no direction of effect. Permutation importance is a more reliable alternative.

B. Questions about the graphs

G1. What does the feature importance bar plot show?
The contribution of each feature to reducing impurity across all trees, normalised to sum to 1. Long bars (for example worst perimeter, worst concave points) mean influential features.

G2. Can importance tell the direction of the effect?
No, only the strength.

G3. What does the model comparison bar chart show?
Test metrics for each model. The Dummy is lowest; the ensembles are slightly higher than the baseline tree in accuracy and clearly higher in AUC, meaning combining many learners improves ranking quality. Be honest that accuracy differences on a 143-sample test set are small.

G3b. Your Random Forest had the same accuracy as the single tree. Does that mean it is no better?
Not necessarily. On one 143-sample test split, accuracy is noisy (one patient = 0.7%). The Random Forest has a much higher AUC (0.995 vs 0.947) and a higher cross-validated accuracy (0.953 vs 0.926), which are more reliable measures.

G4. What does the accuracy-vs-number-of-trees plot show?
For Random Forest accuracy rises then plateaus, so more trees does not overfit. For boosting, test accuracy plateaus (and can drop with too many rounds) while training accuracy keeps rising.

G5. How do the confusion matrices of the ensembles compare with the tree?
Fewer off-diagonal entries (errors), especially fewer false negatives.

G6. What does the cross-validation boxplot show?
The distribution of accuracy across folds. Higher and narrower boxes mean better and more stable models. In my run the single tree is lowest (0.926) and AdaBoost is highest and narrowest (0.977); Random Forest and Gradient Boosting have higher means than the tree but not a clearly smaller spread.

G7. What do the ROC curves show?
The trade-off between true positive rate and false positive rate for each model; curves nearer the top-left (larger AUC) are better. The tree's curve is blocky because a tree gives few distinct probabilities.

C. Code questions

C1. Why `stratify=y`?
To keep the same class proportion in train and test.

C2. What does `n_estimators` mean?
The number of trees in the ensemble.

C3. What does `predict_proba` return for a forest?
The average of the class probabilities predicted by all trees.

C4. Why is `random_state` set?
Bootstrap sampling and feature selection are random; the seed makes results reproducible.

C5. What does `DummyClassifier(strategy='most_frequent')` do?
Always predicts the most common training class. It gives the minimum performance any real model should beat.

Practical 11: Reinforcement Learning with Q-Learning (Grid World)

A. Concept questions

Q1. What is reinforcement learning?
A learning paradigm where an agent interacts with an environment, takes actions, receives rewards, and learns a policy that maximises cumulative reward.

Q2. Define agent, environment, state, action, reward and policy.
Agent: the learner. Environment: the world it acts in. State: the current situation. Action: a choice the agent makes. Reward: scalar feedback. Policy: a mapping from states to actions.

Q3. What is Q-Learning?
A model-free, off-policy, value-based RL algorithm that learns the action-value function $Q(s, a)$: the expected total future reward for taking action a in state s and then acting optimally.

Q4. Explain the Q-learning update rule.
New Q = old Q + α × (reward + γ × best next Q − old Q). The bracket is the temporal-difference (TD) error: the gap between the target and the current estimate.

Q5. What is the learning rate α?
How much new information overrides old. 0 means no learning; 1 means only the latest experience is used. Typical value about 0.1.

Q6. What is the discount factor γ?
Controls how much future rewards matter. Near 0: short-sighted; near 1: far-sighted. Typical 0.9 to 0.99.

Q7. What is exploration vs exploitation?
Exploration: trying new actions to discover better rewards. Exploitation: using current knowledge to pick the best-known action. Both are needed.

Q8. What is epsilon-greedy?
With probability ε choose a random action (explore); with probability 1 − ε choose the action with the highest Q (exploit). ε is usually decayed over time.

Q9. Why decay epsilon?
Early on, the Q-table is uninformed, so explore a lot. Later the policy is good, so exploit more and earn higher rewards.

Q10. What does "model-free" mean?
The agent does not know or learn the environment's transition probabilities or reward function; it learns directly from experience.

Q11. What does "off-policy" mean?
Q-learning updates using the greedy best next action (max Q), regardless of which action the behaviour policy actually takes. SARSA is on-policy: it uses the action actually taken.

Q12. Q-Learning vs SARSA?
Q-learning's target uses $\max_{a'} Q(s', a')$, so it learns the optimal policy and is more optimistic (it takes risks near pits). SARSA uses the $Q(s', a')$ of the action actually taken, so it learns a safer policy that accounts for its own exploration.

Q13. What is the Q-table?
A table with one row per state and one column per action, storing Q-values.

Q14. What is the optimal policy?
The policy that maximises expected cumulative reward from every state; derived as $\pi^*(s) = \arg\max_a Q(s,a)$.

Q15. What is the Bellman equation?
$Q^*(s,a) = E[\, r + \gamma \max_{a'} Q^*(s',a') \,]$: the value of a state-action equals the immediate reward plus the discounted best value of the next state. Q-learning is a sample-based way to solve it.

Q16. What are the limitations of tabular Q-learning?
It does not scale to large or continuous state spaces (the table explodes). Solution: function approximation such as Deep Q-Networks (DQN).

Q17. What is an episode?
One complete run from the start state until a terminal state (goal or pit) or a step limit.

Q18. What is a Markov Decision Process (MDP)?
The mathematical framework for RL: states, actions, transition probabilities, rewards and a discount factor; the next state depends only on the current state and action (Markov property).

Q19. Does Q-learning always converge?
In the tabular case, yes to the optimal Q-values, provided every state-action pair is visited infinitely often and the learning rate decays appropriately.

Q20. Why is the step reward −1?
It penalises long paths, so the agent prefers the shortest route to the goal instead of wandering.

Q21. Why are pits given −10 and the goal +10?
Terminal rewards signal good and bad outcomes. Falling in a pit ends the episode with a big penalty, so the agent learns to avoid it; reaching the goal gives a big reward that propagates backwards through Q-values.

Q22. What is the difference between reward and value (Q)?
Reward is the immediate feedback for one step. Value/Q is the expected sum of all future (discounted) rewards.

Q23. What is the credit assignment problem?
Deciding which earlier actions deserve credit for a delayed reward. Q-learning solves it by propagating value backwards through the TD updates.

Q24. How does RL differ from supervised learning?
There are no labelled examples; feedback is a delayed, evaluative reward signal, and the agent's actions affect the data it sees later.

Q25. What is a terminal state?
A state where the episode ends (goal or pit); its future value is 0, so the update there uses only the immediate reward.

Q26. Real-world uses of RL?
Games (Go, Atari), robotics, recommendation systems, resource scheduling, autonomous driving, and tuning language models from human feedback.

B. Questions about the graphs

G1. What does the learning curve (reward vs episode) show?
Learning progress. It starts low and noisy (exploration, many pit falls), rises as Q-values improve, and flattens near the maximum when the policy has converged.

G2. Why is the curve noisy even at the end?
Because epsilon never reaches 0 (minimum 0.05), the agent still takes occasional random actions. A moving average smooths it.

G3. Why does the reward level off around +3 and not +10?
The goal gives +10 but each of the 7 earlier steps costs −1, so the best possible return is 10 − 7 = +3 on the shortest path.

G4. How do you read the policy grid?
Each cell shows the best action (arrow) from that state, i.e. argmax Q(s, a). Following the arrows from the start should reach the goal while avoiding pits.

G5. What does the value heatmap show?
The maximum Q-value in each state. Values are highest next to the goal and fall with distance (discounting); cells next to pits are lower.

G6. What does the epsilon decay plot show?
Epsilon falling from 1.0 to 0.05: mostly exploration at first, mostly exploitation at the end.

G7. Compare the curves for different epsilon strategies.
A very high fixed epsilon (0.5) learns a correct Q-table but keeps getting low rewards (about -10) because it keeps acting randomly. A small fixed epsilon (0.05) and a decaying epsilon both end near +2 (the best possible is +3), and the decaying one is worst at the start because it begins with 100% random moves. The greedy agent with Q starting at 0 also learns well (optimistic initialisation), but a greedy agent with a pessimistic start (-20) never reaches the goal.

G7b. Your greedy agent (epsilon = 0) learned the optimal policy with no exploration. Does that mean exploration is useless?
No. It worked only because Q was initialised to 0 while every step costs -1: untried actions look better than tried ones (optimistic initialisation), which forces the agent to try them. With a pessimistic initial Q-table (-20) the same greedy agent never reaches the goal. In general, and especially with positive or sparse rewards, an agent needs deliberate exploration such as epsilon-greedy.

G8. What does the steps-per-episode plot show?
Episodes become shorter as the agent learns the direct route, ending near the shortest path length (8 moves).

G9. What does the success-rate curve show?
The fraction of episodes that reach the goal; it rises toward 1 as the policy improves.

G10. What do the alpha and gamma plots show?
Alpha controls learning speed: alpha = 0.01 is much slower, while 0.1 to 0.9 all reach the same final reward in this deterministic grid (a large alpha would be noisy in a stochastic environment). Gamma = 0 makes the agent short-sighted so it cannot learn the path to the goal; gamma of 0.5 and above lets the goal reward propagate back.

G11. What is optimistic initialisation?
Starting Q-values higher than the true values, so every untried action looks attractive and the agent explores them even when it acts greedily. In this grid, Q = 0 is optimistic because real returns are negative (-1 per step).

C. Code questions

C1. What does `Q[s][a] += alpha * (r + gamma * Q[ns].max() - Q[s][a])` do?
The Q-learning (Bellman/TD) update for the state-action just taken.

C2. Why is the next-state value set to 0 when `done`?
A terminal state has no future, so the target is just the immediate reward.

C3. Why break ties randomly in exploitation?
At the start all Q-values are 0, so `argmax` would always pick action 0 and bias exploration; random tie-breaking avoids that.

C4. Why `max_steps = 100`?
To stop endless episodes if the agent wanders without reaching a terminal state.

C5. Why average over several random seeds in Step 5?
RL results are noisy because of random exploration; averaging gives a fairer comparison.